

ESISAR - Examen IN331

Examen de Programmation C

Thomas Molines & Jean-Luc Koning
16 janvier 2026 - Durée 2h

Recommandations Générales

Consignes

Aucun document n'est autorisé à l'exception de ceux fournis sous forme électronique avec cet énoncé :

1. Fichier des consignes préliminaires pour effectuer cet examen : *consignesEleves-amont.pdf*

Nous vous recommandons de suivre scrupuleusement les recommandations de ce document.

2. Fichier des rappels de la syntaxe du langage C : *CheatSheet-LangageC.pdf*

La compréhension du sujet faisant partie de l'épreuve, il ne sera répondu à aucune question. S'il vous semble qu'une erreur s'est glissée dans le sujet, indiquez comment vous la corrigez et répondez à la question en en tenant compte.

Barème

Le barème est donné à titre indicatif. Il pourra évoluer à la marge.

Conseils pour réussir cet examen

1. **Lire** (et comprendre) la totalité des documents avant de vous lancer dans la résolution de l'énoncé.
2. Commencer par vérifier que la commande `make` fonctionne correctement. Ça doit compiler dès le départ.

Faites en sorte de ne jamais trop vous éloigner d'un état dans lequel vous êtes en mesure de compiler. Attention : À la fin de l'épreuve, le projet rendu doit être **"compilable" et "exécutable" !**

3. Implémenter des blocs de code simples et tester tous les cas de figure possibles.

Fichiers fournis

La totalité des fichiers du projet à rendre se trouvent dans l'archive `tar.gz` ci-joint.

- Le fichier `readme.md` décrit en anglais l'énoncé des différents exercices à réaliser. La version en français est donnée dans la suite du présent document.
- Le fichier `Makefile` sert à gérer la construction du projet. Il contient les règles et les dépendances pour compiler vos fichiers sources et générer l'exécutable. Il permet de produire le fichier exécutable qui sera dans `out/bin/` ainsi que la bibliothèque dans `out/lib/`. Le fichier `Makefile` est *pour lecture seule*. **Il ne doit pas être modifié !** (Sauf si évidemment certaines options de compilations ne convenaient pas.)
- Le fichier d'entête `inc/functions.h` contient la déclaration des fonctions à écrire, les macros et les définitions partagées par les différents fichiers C. Ce fichier est également *pour lecture seule*. **Il ne doit pas être modifié !**
- Il est demandé d'écrire votre code dans les fichiers `src/*.c`. Il s'agit de l'implémentation des fonctions déclarées dans le fichier d'entête `functions.h`. Les fichiers C sont fournis vides (signature seule sans implémentation). Il vous est demandé de modifier ces fichiers pour réaliser le projet.

Pour rappel, **seuls les fichiers C doivent être modifiés**. Le `Makefile` et les fichiers de header `*.h` doivent être conservés *en lecture seule* afin de garantir l'intégrité du projet.

Sujet de l'examen

Pour cet examen, il vous est demandé d'implémenter les fonctions qui sont détaillées dans le fichier d'entête `inc/functions.h`. Vous y trouverez notamment la spécification des entrées et des sorties de chacune des fonctions attendues.

Définition tirée de Wikipédia:

Une **anagramme** (le mot est féminin) est un mot ou une expression obtenu en permutant les lettres du mot ou de l'expression de départ.

Exercice 1 : Comparaison de deux caractères (2 points)

Un petit exercice pour s'échauffer : Écrire une fonction C `compare_characters` qui compare deux caractères en tenant compte, le cas échéant, de leur casse (majuscule ou minuscule).

Fichier à implémenter et tester : `src/compare_characters.c`

Exercice 2 : Tester si deux mots sont des anagrammes (7 points)

Dans cet exercice, il vous est demandé d'écrire la fonction `is_anagram` qui détermine si deux mots sont des anagrammes.

Fichier à implémenter et tester : `src/is_anagram.c`

Indice : la fonction `is_anagram` se sert de la fonction `compare_characters`.

Exercice 3 : Trouver les anagrammes d'un mot au sein d'un dictionnaire (7 points)

Dans cet exercice, vous devez implémenter la fonction `find_anagram_in_dict` qui parcourt un fichier dictionnaire et affiche, à l'aide d'un `printf`, les anagrammes d'un mot qui sont présent dans ce fichier dictionnaire.

Cette fonction retourne le nombre d'anagrammes du mot trouvées dans le dictionnaire.

Fichier à implémenter et tester : `src/find_anagram_in_dict.c`

Indice : la fonction `find_anagram_in_dict` se sert de la fonction `is_anagram`.

Exercice 4 : Fonction principale (4 points)

Écrire la fonction principale `main` qui analyse les arguments qui lui sont passés et retourne 0 si au moins une anagramme a été trouvé, et 1 sinon.

Fichier à implémenter et tester : `src/anagram.c`

Aide

- La fonction `fgets()` peut être utilisée pour lire une ligne dans le fichier de données. Elle lit jusqu'à ce qu'un caractère "NewLine" ou "End Of File" soit atteint.
- Comme leur nom le laisse supposer, les fonctions `fopen` et `fclose` sont utilisées pour respectivement *ouvrir* ou *fermer* un fichier. Si vous souhaitez ouvrir le dictionnaire en mode *lecture*, utilisez le mode "`r`".

Pour l'ouverture d'un fichier, une valeur de retour `NULL` indique une erreur.

- La commande `man` permet d'accéder au descriptif des fonctions standards. En cas de besoin, voir `man fgets`, `man fopen`, et `man fclose`. Vous y trouverez des façons de les utiliser ainsi que des exemples.